Ingeniería En Electrónica
Y Sistemas Inteligentes

Informe Técnico LMS y SVM

Evaluación 1

NOMBRES:

- Arturo Ignacio Briceño Parra
- Vicktor Thomas Díaz Sepúlveda
- Nicolás Eduardo Sánchez Elera

CARRERA: Ingeniería En Electrónica Y Sistemas
ASIGNATURA: Inteligencia Artificial
PROFESOR: Israel Elías Muñoz Carrera
FECHA:24/04/2026

Ingeniería En Electrónica
Y Sistemas Inteligentes

inacap

Contenido

Informe Técnico LMS y SVM 1

Evaluación 1 1

1 Introducción 3

2 Marco Teórico 4

2.1 LMS 4

2.2 SVM 5

3 Desarrollo 6

3.1 Ejercicio 1 (LMS) 6

3.1.1 Explicación del sistema 6

3.1.2 Código Arduino (explicada línea por línea) 6

Código Python (explicada línea por línea) 7

3.1.3 Gráficos obtenidos 8

3.2 Ejercicio 2 (SVM) 9

3.2.1 Explicación del sistema 9

3.2.2 Código Arduino (explicada línea por línea) 10

3.2.3 Código Python (explicada línea por línea) 11

3.2.4 Resultado de clasificación 12

3.3 Materiales utilizados 13

3.4 Tecnologías utilizadas 13

3.5 Diagramas 14

3.6 Resultados 15

3.6.1 Análisis de comportamiento 15

3.7 Conclusiones 16

3.7.1 Aprendizajes 16

3.7.2 Dificultades 16

3.7.3 Aplicaciones Industriales 16

1 Introducción

1.1 ¿Qué es IA?

La IA, iniciales de Inteligencia Artificial es un conjunto de tecnologías que permite a las computadoras aprender, razonar y realizar una variedad de tareas avanzadas de formas que antes requerían inteligencia humana, como comprender el lenguaje, analizar datos y hasta proporcionar sugerencias útiles para sus tareas asignadas.

Además, la Inteligencia Artificial se basa en disciplinas como el aprendizaje automático, el procesamiento de lenguaje natural y la visión por computadora, lo que le permite adaptarse a distintos contextos y mejorar su desempeño con el tiempo.

es una herramienta muy importante en áreas como la medicina, la industria, la educación y la automatización, facilitando procesos y apoyando la toma de decisiones.

1.2 Aplicación en Electrónica

La inteligencia artificial transforma la electrónica desde la forma en que diseñamos los circuitos hasta cómo funcionan los dispositivos que llevamos en el bolsillo. Básicamente, hemos pasado de circuitos que solo ejecutan tareas predefinidas en codigo a sistemas inteligentes que son capaces de aprender y tomar decisiones acertadas para realizar su tarea de manera eficiente de manera independiente.

En el área de control, la inteligencia artificial permite implementar sistemas más avanzados que los controladores tradicionales como PID. Por ejemplo, se utilizan redes neuronales y algoritmos de aprendizaje por refuerzo para ajustar automáticamente parámetros en tiempo real en motores eléctricos, brazos robóticos o drones. Esto permite mejorar la estabilidad, reducir errores y adaptarse a cambios en el entorno, como variaciones de carga o condiciones externas, algo que los sistemas clásicos no pueden hacer con la misma eficacia.

Otra aplicación importante es el mantenimiento predictivo y el procesamiento de señales en sistemas electrónicos. Mediante el uso de sensores y algoritmos de aprendizaje automático, es posible analizar vibraciones, temperatura o consumo eléctrico para detectar fallas antes de que ocurran. Esto se utiliza en maquinaria industrial, fuentes de poder y sistemas embebidos, permitiendo reducir costos y evitar daños mayores. Además, la IA también se aplica en el procesamiento de imágenes y señales, como en cámaras inteligentes, reconocimiento de patrones o sistemas de comunicación, mejorando la eficiencia y autonomía de los dispositivos electrónicos modernos.

1.3 Objetivo del Trabajo

El objetivo del presente trabajo es diseñar e implementar un sistema embebido basado en Arduino que permita la adquisición de señales en tiempo real y su posterior procesamiento mediante técnicas de inteligencia artificial.

Específicamente, se busca aplicar el algoritmo Least Mean Square (LMS) para el filtrado adaptativo de señales, permitiendo reducir el ruido y analizar su comportamiento dinámico, y el algoritmo Support Vector Machine (SVM) para la clasificación de datos obtenidos experimentalmente.

Además, se pretende integrar la comunicación entre hardware y software mediante el uso de Python, facilitando la visualización gráfica de los resultados y el análisis comparativo entre señales originales y procesadas.

Finalmente, el trabajo tiene como propósito demostrar la utilidad de estos algoritmos en aplicaciones reales de la electrónica y el procesamiento digital de señales, evidenciando cómo la inteligencia artificial puede mejorar el rendimiento y la toma de decisiones en sistemas electrónicos modernos.

2 Marco Teórico

2.1 LMS

- Definición:

LMS que tiene las siglas de Least Mean Square es uno de los algoritmos utilizados en el procesamiento digital de señales y el control automático, Su función principal es actuar como filtro adaptativo de una señal especifica y un sistema que ajusta sus propios parámetros automáticamente para minimizar el error entre la señal que produce y la señal deseada.

- Aplicaciones:

Las aplicaciones principales de LMS en electrónica son:

- **Cancelación de ECO:** Utilizado en tratamiento de audio para evitar que se escuche la propia voz de la persona de regreso o sea cual sea la señal que se necesite evitar el eco.

- **Ecualización de Canales:** Corrige las distorsiones que sufre una señal de internet al viajar por un cable de cobre o fibra óptica, existen muchas formas de tratar las señales y esta es de las más usadas globalmente.

- **Reducción de Ruido en Medicina:** Para limpiar señales de ECG (electrocardiogramas) eliminando el ruido de la respiración o de los músculos del paciente, en palabras cortas es una tecnología que puede salvar la vida de muchas personas usándola en casos necesarios.

Cabe destacar que existen muchos más tipos de usos ya que gran parte de cosas en electrónica usan señales así que es una excelente alternativa para trabajar señales que esten en riesgo de perderse o de degradarse en el tiempo.

2.2 SVM

-Definición:

SVM siglas de Support Vector Machine es un algoritmo de aprendizaje automático supervisado que se utiliza tanto para desafíos de clasificación como de regresión, aunque su uso más común es la clasificación. las SVM son especialmente valoradas porque son menos propensas al overfitting que otros algoritmos. Donde Overfitting o sobreajuste se refiere a cuando un modelo de aprendizaje automático aprende demasiado bien los datos de entrenamiento, pero de una forma incorrecta para generalizar.

- Clasificación:

Debido a su alta precisión y capacidad para manejar datos de alta dimensión, las SVM se aplican en campos muy diversos:

- **Categorización de Texto:** Diferenciar entre noticias, correos (spam vs. no spam) o análisis de sentimiento.

- **Categorización de Bioinformática:** Se utiliza para clasificar secuencias de ADN o compuestos químicos, donde las características suelen ser miles.

- **Reconocimiento de Imágenes y Visión Artificial:** Identificar partes del rostro o expresiones usado bastante en sistemas de video vigilancia e investigación, incluso en uso de policías y procesos de identificación personal.

- **Reconocimiento de Escritura a Mano:** Se emplean ampliamente para reconocer caracteres escritos manualmente (como códigos postales en sobres), convirtiendo trazos visuales en datos digitales legibles.

3 Desarrollo

3.1 Ejercicio 1 (LMS)

3.1.1 Explicación del sistema

El sistema consiste en un circuito simple basado en la adquisición de datos desde un potenciómetro de 1 kOhm. Este se conecta a un microcontrolador Arduino, el cual entrega un voltaje de 5V, una señal de salida y conexión a tierra. A medida que el usuario modifica la posición del potenciómetro, el microcontrolador lee valores analógicos en un rango de 0 a 1023, representando diferentes niveles de voltaje.

Al registrar estos valores en función del tiempo, se obtiene una señal variable que presenta cambios dependiendo del movimiento del usuario. Estos datos son recopilados durante un periodo de 30 segundos y posteriormente enviados a un computador mediante comunicación serial a través de USB, utilizando un puerto COM o TTY según el sistema operativo.

Finalmente, la señal es procesada mediante un filtro LMS implementado en Python, el cual permite analizar y visualizar gráficamente tanto la señal original como la señal filtrada, facilitando la comprensión del comportamiento del sistema.

3.1.2 Código Arduino (explicada línea por línea)

Código De Arduino Leonardo Usado Para Obtener información del potenciómetro

```
// Definimos el pin ANALOG IN del potenciómetro
const int potPin = A0;
//el codigo que se generara para hacer la conexion.
void setup() {
  // El Arduino Leonardo usa Serial.begin() pero la comunicación USB es nativa
  // 9600 es la velocidad estándar que configuramos en Python
  Serial.begin(9600);

  // Esperar a que el puerto serial esté listo (específico para Leonardo)
  while (!Serial) {
    ;
  }
}
//Crea un ciclo que se repetiravbnnbj
void loop() {
  // 1. Leer el valor del potenciómetro (0 a 1023)
  int valorPot = analogRead(potPin);

  // 2. Enviar el valor por el puerto serial
  // Usamos println para que cada dato vaya en una línea nueva
  Serial.println(valorPot);

  // 3. Pequeña pausa para no saturar el buffer
  // Esto nos da aproximadamente 100 lecturas por segundo
  delay(10);
}
```

Código Python (explicada línea por línea)

Código Python LMS.py Exportado De Visual Studio Code

LMS.py

```python
import serial # permite la comunicación con el puerto serial
import numpy as np #Librería para manejo de arrays y operaciones matemáticas
import matplotlib.pyplot as plt #Librería para graficar los datos
import time #Librería para manejar el tiempo

# 1. Configuración del Puerto Serial
puerto_serial = serial.Serial('COM8', 9600, timeout=1) #timeout=1 para evitar bloqueos si no hay datos disponibles.
time.sleep(2) # Esperar un momento para que el puerto serial se estabilice al inicio.

# 2. Variables de control
duracion_segundos = 30  # Duración total de la captura en segundos
mu = 0.03  # Ahora 0.03 es seguro porque normalizaremos la entrada
w = 0.0     # Peso inicial del filtro LMS, se puede iniciar en 0 o con un valor pequeño
datos_originales = []# Lista para almacenar los valores originales del potenciómetro (0-1023) para graficar
posteriormente.
datos_filtrados = []# Lista para almacenar los valores filtrados por el algoritmo LMS, que se escalarán de vuelta a
0-1023 para la gráfica.
tiempos = []# Lista para almacenar los tiempos de cada lectura, lo que permitirá graficar la evolución de la señal a
lo largo del tiempo.

print(f"Iniciando captura por {duracion_segundos} segundos ...")#Mensaje que indica que inicio la captura de datos
inicio_test = time.time()#Marca el tiempo de inicio de la captura para poder controlar la duración total del
proceso.

try: #Bucle principal de captura y procesamiento de datos. Se ejecutará hasta que se alcance la duración
especificada en duracion_segundos.
#Mientras el tiempo transcurrido desde el inicio de la captura sea menor que la duración especificada, se seguirá
leyendo y procesando datos del puerto serial.
    while (time.time() - inicio_test) < duracion_segundos:
        ##Lee una línea del puerto serial, la decodifica de bytes a string usando UTF-8, y elimina cualquier espacio
en blanco o caracteres de nueva línea.
        linea = puerto_serial.readline().decode('utf-8').strip()
        if linea:#Si la línea leída no está vacía, se procede a procesarla.
            try:
                # --- NORMALIZACIÓN ---
                # Convertimos 0-1023 a un rango de 0.0 a 1.0
                valor_crudo = float(linea) #Convierte la cadena leída del puerto serial a un número de punto
flotante.
                x_n = valor_crudo / 1023.0 #Normaliza el valor del potenciómetro dividiéndolo por 1023.0
                #inclusive en las primeras versiones del codigo el valor del potenciometro subia hasta el infinito
                # --- Algoritmo LMS ---
                y_n = w * x_n          # Salida estimada
                e_n = x_n - y_n        # Error
                w = w + mu * e_n * x_n # Actualización del peso

                # --- GUARDAR DATOS (Volvemos a escalar a 1023 para la gráfica) ---
                datos_originales.append(valor_crudo)# almacena el valor original del potenciometro sin normalizar
                datos_filtrados.append(y_n * 1023.0)# almacena el valor filtrado por el algoritmo LMS
                tiempos.append(time.time() - inicio_test)# Almacena el tiempo transcurrido desde el inicio de la
captura

            except ValueError: #si el Arduino envía un mensaje de error o datos corruptos, se captura la excepción
ValueError y se ignora esa línea
                continue#Si ocurre un error al convertir la línea a un número, se ignora esa línea y se continúa con
la siguiente iteración del bucle

finally: #Cerrar el puerto serial al finalizar la captura, ya sea por alcanzar la duración especificada o por una
interrupción.
    puerto_serial.close()#Cierra el puerto
    print("Captura finalizada y puerto cerrado.") #mensaje indicando al usuario que la captura termino

# 4. Graficación
plt.figure(figsize=(12, 6)) #crea una figura tamaño 12×6
# Señal original en Cyan y Filtrada en Azul como se pidio como requisito
plt.plot(tiempos, datos_originales, color="cyan", label="Señal Original (Potenciómetro)")#grafica la señal original
plt.plot(tiempos, datos_filtrados, color="blue", label="Señal Filtrada (LMS)")#grafica la señal filtrada por el
algoritmo LMS

plt.xlabel("Tiempo (s)")#etiqueta del eje X
plt.ylabel("Valor Potenciómetro (0-1023)")#etiqueta del eje
plt.title("Procesamiento de Señal en Tiempo Real: Arduino + LMS") #titulo de la gráfica
plt.legend()#Agrega una leyenda para identificar las líneas de la gráfica
plt.grid(True, linestyle='--', alpha=0.6)#agrega la cuadrilla

# Ajustar los límites del eje Y para que no se vea infinito si algo sale mal
plt.ylim(-50, 1100)
# Mostrar la gráfica
plt.show()
```

3.1.3 Gráficos obtenidos

Grafico N1	Grafico N2

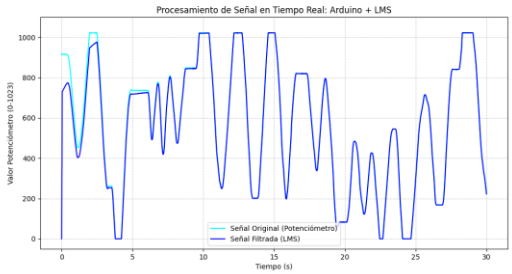
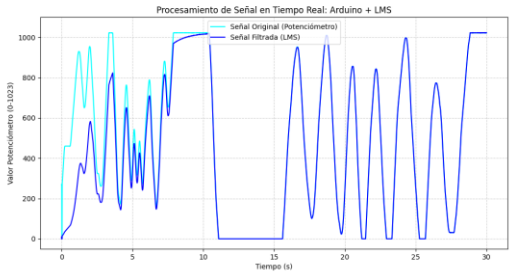

	

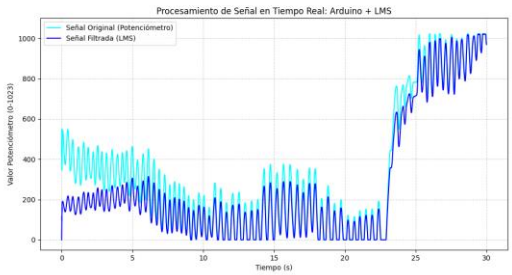
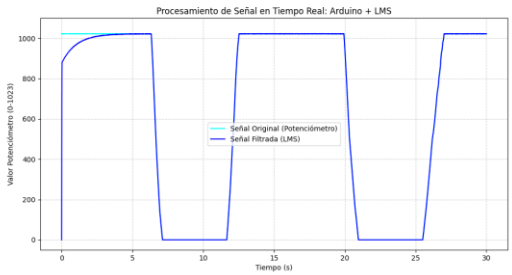
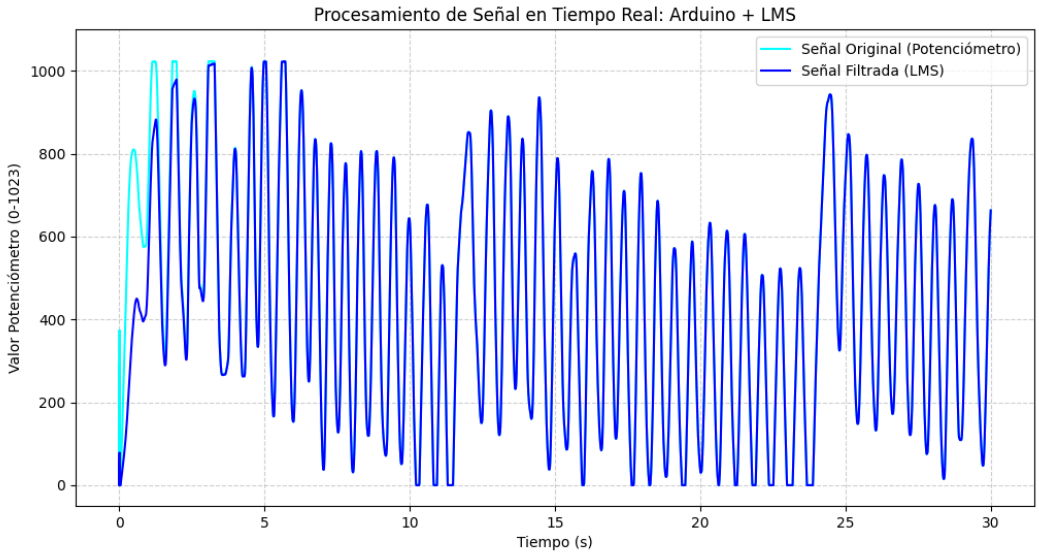
Grafico N3	**Grafico N4**
	

Grafico De cambio de señal Extremo con mu=0.03



3.2 Ejercicio 2 (SVM)

3.2.1 Explicación del sistema

El sistema opera mediante un esquema de comunicación bidireccional entre un script de Python y un microcontrolador Arduino Uno R3. El circuito físico consta de un botón tipo switch ensamblado en una protoboard, conectado al pin analógico A0 del microcontrolador. Este pin está configurado mediante software como una entrada digital (GPIO) y utiliza una resistencia Pull-Down física para garantizar que el estado lógico en reposo sea de 0V (LOW).

El flujo operativo se divide en dos fases principales: Captura Sincronizada y Clasificación SVM.

Durante la fase de captura, el computador solicita al usuario ingresar por consola un valor numérico entero, restringido a un rango de 0 a 30. Este dato se codifica y se transmite vía USB (Serial a 9600 baudios) hacia el Arduino. El microcontrolador recibe la información, limpia el búfer de memoria para evitar lecturas residuales y entra en un estado de bloqueo reactivo, esperando una confirmación física. Para validar el dato, el usuario debe presionar el botón en el circuito. El programa de Arduino incorpora un retardo (delay) de 200 milisegundos que actúa como un filtro antirrebote (debouncing) por software, aislando el ruido de la señal del botón. Al soltar el pulsador, el Arduino transmite un token de confirmación estructurado (DATO:XX) de regreso al computador.

Una vez que el sistema recopila exitosamente un dataset de 10 muestras confirmadas, inicia la fase de procesamiento. Utilizando la biblioteca Scikit-learn, los datos se formatean en un arreglo matricial y se etiquetan en dos categorías predefinidas: Clase 1 (valores entre 0 y 14) y Clase 2 (valores entre 15 y 30).

Posteriormente, se entrena un modelo de Máquinas de Vectores de Soporte (SVM) configurado con un kernel lineal. El algoritmo matemático extrae el peso (w) y el sesgo (b) para calcular algebraicamente la frontera de decisión unidimensional mediante la ecuación:

$$x = -b / w$$

Finalmente, se utiliza la librería Matplotlib para generar una representación visual del modelo. Al tratarse de un espacio de una dimensión, las muestras capturadas se proyectan uniformemente sobre el eje X (forzando la coordenada Y a cero), separando visualmente ambas clases mediante una línea que marca la frontera de decisión exacta calculada por la inteligencia artificial.

3.2.2 Código Arduino (explicada línea por línea)

```
const int buttonPin = A0; // asigna el pin analogico A0 para que funcione como entrada digital del boton
int count = 0;            // crea un contador iniciando en cero para llevar el registro de las 10 muestras
int inputNumber = 0;      // crea una memoria temporal para guardar el numero que llega desde la PC

void setup() {
  Serial.begin(9600);     // enciende el puerto USB a una velocidad de 9600 bits por segundo

  // configura el pin A0 para recibir corriente al ser Pull-Down asume que le pusiste una resistencia a tierra
  pinMode(buttonPin, INPUT);

  Serial.println("Sistema SVM Iniciado."); // envia un mensaje inicial a la computadora
}

void loop() {
  if (count < 10) { // revisa si el contador aun es menor a 10 si ya llego a 10 ignora todo este bloque

    Serial.println("Ingrese un numero entre 0 y 30"); // pide el dato al usuario

    // el arduino se queda congelado en esta linea dando vueltas hasta que detecte que llego informacion por el cable
    while (Serial.available() == 0) {}

    // lee el texto que llego por el USB y lo convierte matematicamente en un numero entero
    inputNumber = Serial.parseInt();

    // limpieza de memoria temporal: borra la "basura" invisible que quedo flotando
    while(Serial.available() > 0) { Serial.read(); }

    if (inputNumber >= 0 && inputNumber <= 30) { // filtro de seguridad: verifica que el numero este en el rango correcto
      Serial.print("Ingreso el numero: ");       // avisa que numero entendio el Arduino
      Serial.print(inputNumber);
      Serial.println(". Presione boton para confirmar");

      // 1. congela el arduino aqui mientras el pin lea LOW ws decir espera a que PRESIONES el boton
      while (digitalRead(buttonPin) == LOW) {}

      // 2. pausa el microprocesador 200 milisegundos para ignorar las vibraciones fisicas del resorte del boton
      delay(200);

      // 3. congela el arduino aqui mientras el pin lea HIGH es decir espera a que SUELTES el boton
      while (digitalRead(buttonPin) == HIGH) {}

      // 4. envia por el cable la palabra clave exacta para que el script de Python en la PC la intercepte
      Serial.print("DATO:");
      Serial.println(inputNumber); // envia el numero que validamos

      count++; // le suma 1 al contador de muestras

    } else {
      // si el numero no estaba entre 0 y 30 arroja este error y el bucle vuelve a empezar sin sumar nada al contador
      Serial.println("Error: Fuera de rango.");
    }
  }
}
```

3.2.3 Código Python (explicada línea por línea)

SVM.py

```python
import serial #sirve para comunicarse con el puerto serial mediante los pines de transmision (TX/RX)
import time #con esta libreria puedo tengo acceso al reloj interno de la CPU
import numpy as np #esta bliblioeca sirve para procesar arreglos matriciales
import matplotlib.pyplot as plt #gracias a esta libreria podemos graficar y visualizar los resultados
from sklearn import svm #carga los modelos matematicos de machine learning

PUERTO = 'COM3' #define el puerto en el que esta el arduino
BAUD_RATE = 9600 #define la velocidad del reloj

datos_capturados = [] #define un arreglo vacio para guardar las mustras

try:
    ser = serial.Serial(PUERTO, BAUD_RATE, timeout=1) #con este comando pedimos control del puerto
    time.sleep(2) # Esperar a que el Arduino se reinicie tras la conexion
    print("--- Sistema de Captura SVM ---")

    while len(datos_capturados) < 10: #se repite 10 veces
        # 1. Ingreso de datos en el PC
        numero = input(f"\n[{len(datos_capturados)+1}/10] Ingrese numero (0-30): ")

        if not numero.isdigit() or not (0 <= int(numero) <= 30): #aqui se flitran letras o numeros fuera del rango
            print("Error: Ingrese un numero valido entre 0 y 30.")#si el numero no es valido, se muestra un mensaje de error
            continue#si el numero es valido, se convierte a entero para su posterior uso

        # 2. Enviar el numero al Arduino
        ser.write(f"{numero}\n".encode('utf-8')) #traduce los caracteres del numero a valores binarios
        print("Enviado. Ahora presione el BOToN FiSICO en el Arduino para confirmar...")

        # 3. Esperar la confirmacion del Arduino (DATO:XX)
        confirmado = False
        while not confirmado:
            #lee el registro para saber si el arduino envio informacion de vuelta
            if ser.in_waiting > 0:
                #lee y decodifica la informacio que viene a texto y elimina los espacios o saltos de linea en los extremos
                linea = ser.readline().decode('utf-8').strip()
                #es basicamente un filtro que solo prosesa la infomracion si resive el codigo de confirmacion
                if linea.startswith("DATO:"):
                    valor = int(linea.split(":")[1]) #corta el texto por la mitad usando los dos puntos como tijera
                    datos_capturados.append(valor)#guarda el valor confirmado por el arduino en el arreglo de datos capturados
                    print(f"Confirmado! El Arduino registro: {valor}")
                    confirmado = True#sale del bucle de espera para el siguiente numero
                else:
                    # Imprime mensajes informativos del Arduino (como "Presione boton...")
                    print(f" [Arduino]: {linea}")#si el arduino envia otro mensaje que no es el de confirmacion, se imprime en
la consola para que el usuario sepa que esta pasando

    ser.close()
# Si ocurre cualquier error durante la comunicacion o el proceso
except Exception as e:
    print(f"Error critico: {e}")#si algo sale mal, se imprime el error en la consola para que el usuario sepa que paso y se
cierra el puerto si es necesario
    exit()# Si todo sale bien, se cierra el puerto al finalizar la captura de datos

# --- PROCESAMIENTO Y GRaFICA ---
X = np.array(datos_capturados).reshape(-1, 1) #obliga al arreglo a transformarse en una columna vertical con 10 filas
y = np.where(X <= 14, 0, 1).ravel() #esta parte clasifica si el valor pertenece al grupo uno o al grupo dos

# Entrenamiento del modelo
modelo = svm.SVC(kernel='linear') #esto le dice al modelo que trace una linea recta sin hacer curvas
modelo.fit(X, y) #aqui se inyecta la matriz X y el vector y para calcular la separacion optima de los valores ingresados

# Calculo de la frontera
peso = modelo.coef_[0][0]
#el sesgo es el valor que se suma a la multiplicacion del peso por la entrada para determinar la clase a la que pertenece un
nuevo dato
sesgo = modelo.intercept_[0]
#esta formula sale del despeje de la formula w*x+b=0 queando asi x=-b/w
frontera = -sesgo / peso

# Generacion de la grafica
plt.figure(figsize=(10, 5)) #crea un lienzo en blanco para dibujar de 10x5
#filtra solo los datos del grupo 1 los aplasta contra el piso Y=0 y los dibuja como circulos de color cyan
plt.scatter(X[y==0], np.zeros_like(X[y==0]), color='cyan', edgecolors='black', s=150, label='Clase 1 (Bajo)')
#filtra solo los datos del grupo 2 los aplasta contra el piso Y=0 y los dibuja como circulos de color azul
plt.scatter(X[y==1], np.zeros_like(X[y==1]), color='blue', edgecolors='black', s=150, label='Clase 2 (Alto)')
#dibuja una linea recta vertical de color rojo justo en el numero que calculo la frontera
plt.axvline(x=frontera, color='red', linestyle='--', linewidth=3, label=f'Frontera SVM: {frontera: 2f}')
#configura los detalles de la grafica como el titulo, etiquetas, leyenda y cuadrilla
plt.title('Clasificacion SVM: Datos de Arduino Confirmados por Boton')
plt.xlabel('Valor de Entrada')#etiqueta del eje X
plt.yticks([])#elimina las marcas del eje Y para que se vea mas limpio
plt.grid(True, alpha=0.3)#agrega una cuadrilla con transparencia para facilitar la lectura de la grafica
plt.legend()#agrega una leyenda para identificar las clases y la frontera de decision
#imprime el resultado de la frontera de decision en la consola
print(f"\nProceso terminado. Frontera de decision calculada en: {frontera: 2f}")
plt.show()#muestra la grafica al usuario
```

3.2.4 Resultado de clasificación

Grafico N1	Grafico N2

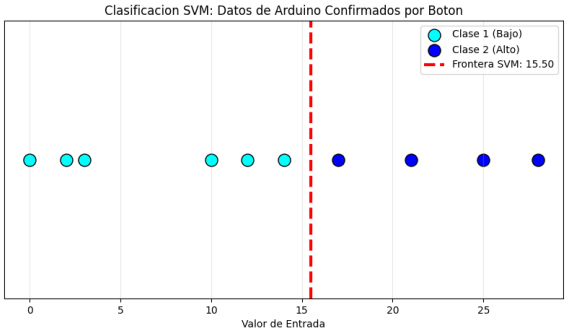
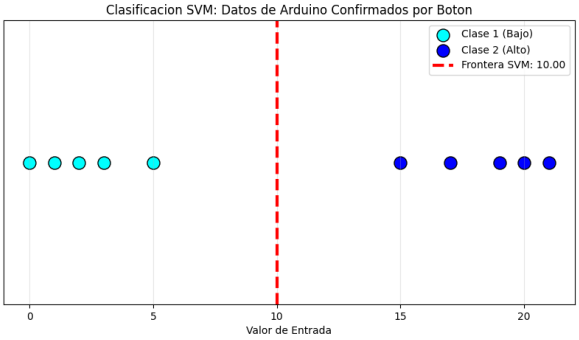
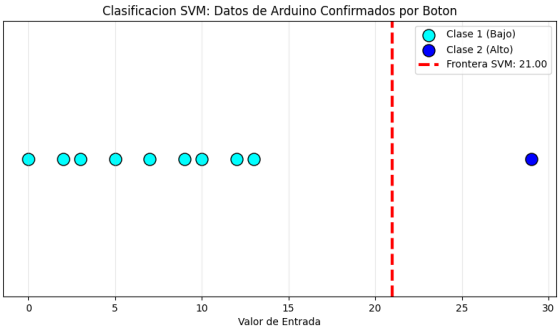
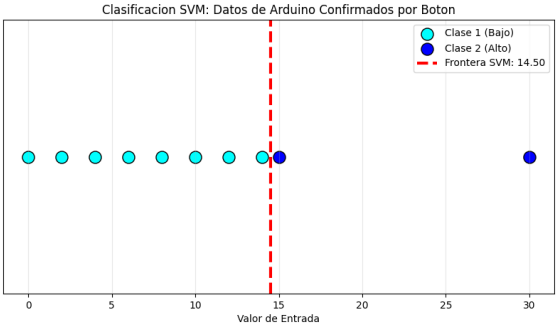

	

Grafico N3	**Grafico N4**
	

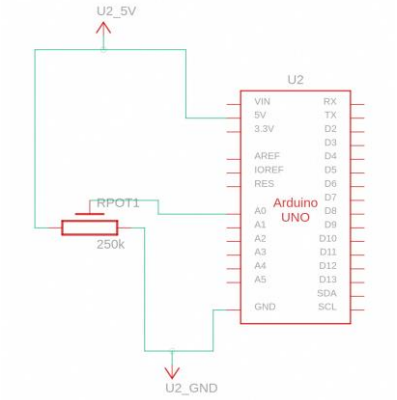
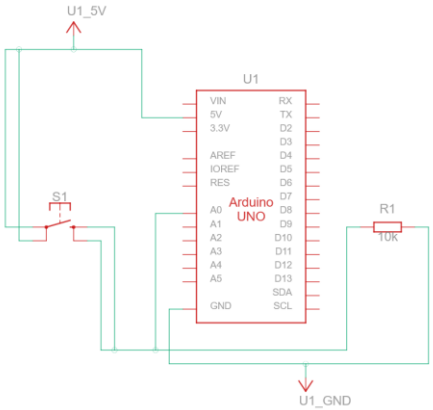
3.3 Materiales utilizados

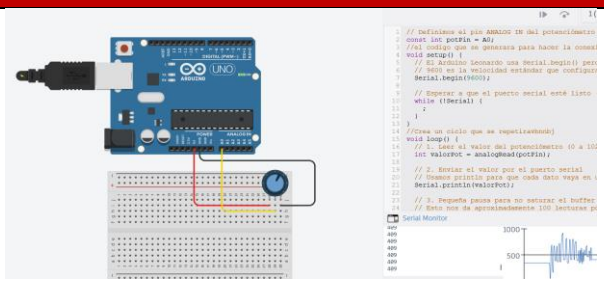
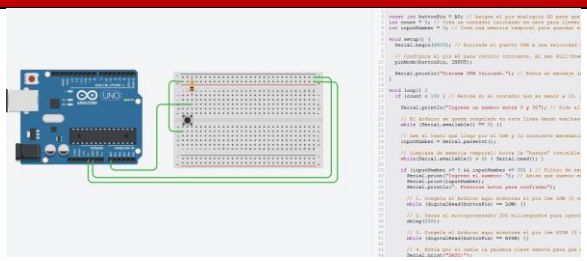
- Arduino Uno R3 y Arduino Uno Leonardo
- Potenciometro 1K Ohm
- Resistencia de 10K Ohm
- Botón Tipo Switch
- Protoboard
- Cables Dupont

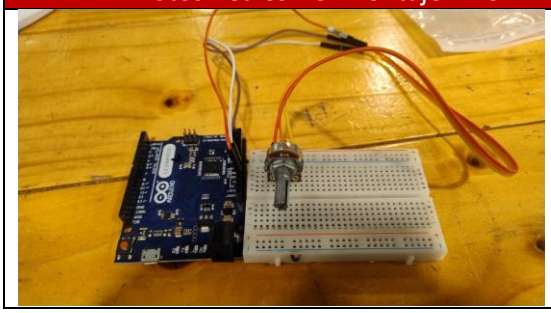
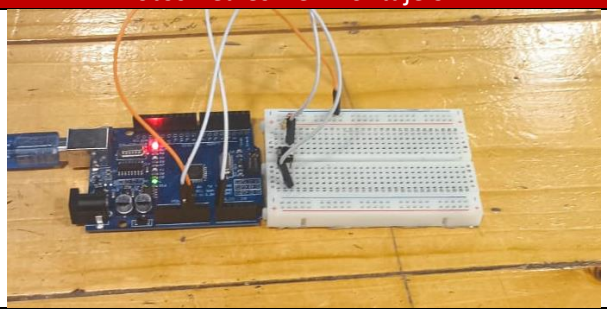
3.4 Tecnologías utilizadas

- Arduino IDE De Arduino
- Visual Studio Code De Microsoft
- Microsoft Office Word De Microsoft
- Python Con las librerías:
    - Numpy
    - Mathplotlib
    - Scikit-learn
    - serial

3.5 Diagramas

Plano Electrónico y esquema de conexión LMS	Plano Electrónico y esquema de conexión SVM
	

Simulación En Thinkercad LMS	Simulación En Thinkercad SVM
	

Fotos Reales Del Montaje LMS	Fotos Reales Del Montaje SVM
	

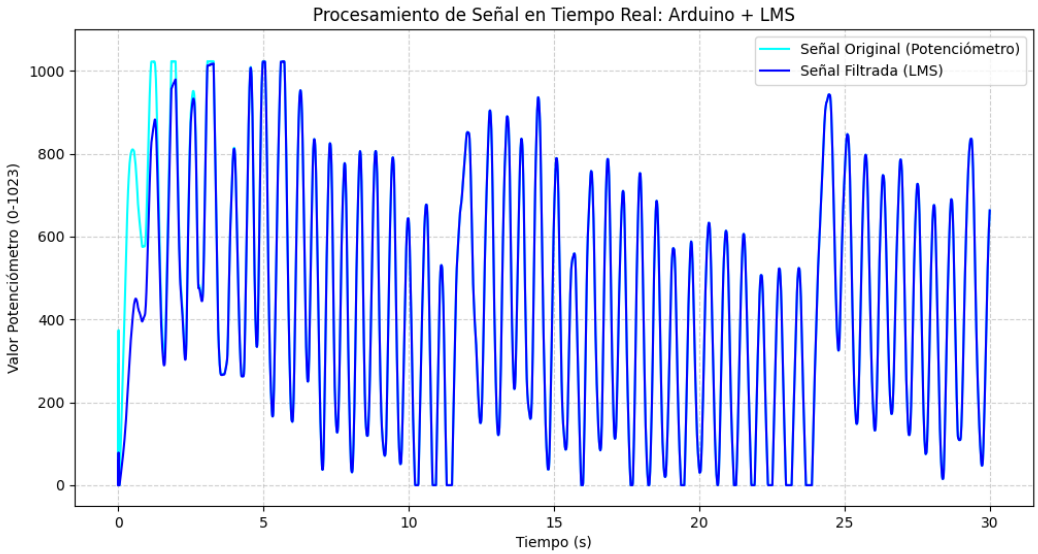
3.6 Resultados

3.6.1 Análisis de comportamiento

3.6.1.1 Comportamiento LMS

Como se puede Observar en los gráficos anteriores se puede ver una clara tasa de aprendizaje en los primeros segundos de la señal para cuando pasan unos segundos la señal filtrada tenga un gran parecido a la original, esto es muy útil en sistemas de filtrado de señales, cabe destacar que a través de pruebas empíricas se pueden lograr mejores resultados, pero estos valores fueron elegidos para permitir la observación del aprendizaje en tiempo real.

3.6.1.2 Comparación Señal Original VS Filtrada



La siguiente imagen nos permite observar como la señal original de color celeste inicia separada de la señal azul que es la señal filtrada donde podemos observar que mientras más avanza el valor del Tiempo(s) nuestra señal filtrada más se acerca ya que va aprendiendo activamente hasta volverse en extremo similar a la original.

En resumen: es la “verdad” contra la señal que comparas

Mientras que la otra señal generada por el algoritmo LMS

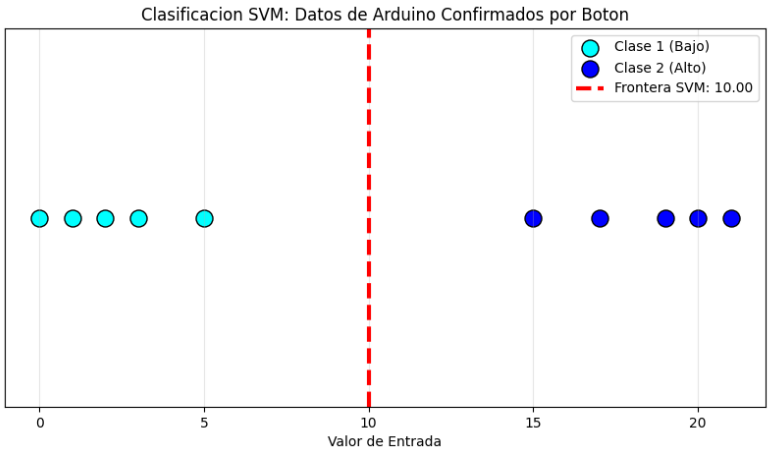
- Es una estimación de la señal original.
- Se obtiene aplicando un filtro adaptativo a la entrada.
- Va cambiando con el tiempo (aprende)

3.6.1.3 Comportamiento SVM

Como se ilustra en las gráficas, el algoritmo SVM clasifica los grupos de datos mediante un hiperplano de separación. Esta frontera se calcula optimizando la distancia respecto a los puntos más próximos de cada clase, denominados vectores de soporte. Cabe destacar que el resto de los datos no influyen en la definición del hiperplano, el cual se sitúa de forma equidistante a los vectores de soporte para maximizar el margen de clasificación.

3.6.1.4 Evaluación de Clasificador SVM

Como se evidencia en la figura, la frontera de decisión del modelo SVM se define exclusivamente por los puntos más cercanos entre ambas clases, correspondientes a los valores 5 y 15. Estos puntos críticos actúan como los vectores de soporte del sistema. El algoritmo ubica la frontera de separación exactamente en el punto medio (valor 10), maximizando así el margen de clasificación a 5 unidades de distancia para cada clase. Cabe destacar que el resto de las observaciones no tienen influencia matemática en el cálculo de este límite.



3.7 Conclusiones

3.7.1 Aprendizajes

Durante el desarrollo de este trabajo se logró comprender e implementar conceptos fundamentales de inteligencia artificial aplicados a la electrónica. En particular, se adquirió experiencia en el uso del algoritmo LMS como filtro adaptativo, permitiendo analizar cómo una señal puede ser mejorada mediante la reducción de ruido en tiempo real. Asimismo, se comprendió el funcionamiento del algoritmo SVM para tareas de clasificación, observando cómo es posible separar datos en distintas clases mediante una frontera de decisión.

Además, se fortalecieron conocimientos en la integración de sistemas embebidos con software de alto nivel, utilizando Arduino para la adquisición de datos y Python para su procesamiento y visualización. Esto permitió entender la importancia de la comunicación serial y la interacción entre hardware y software en sistemas inteligentes.

3.7.2 Dificultades

Uno de los principales desafíos fue la correcta sincronización entre Arduino y Python, especialmente en la transmisión y recepción de datos por comunicación serial, donde errores en el manejo del buffer o tiempos de lectura podían generar datos inconsistentes.

Otra dificultad relevante fue la implementación y ajuste de los parámetros del algoritmo LMS, particularmente el valor de mu (tasa de aprendizaje), ya que valores inadecuados podían provocar inestabilidad o una mala convergencia del sistema. En el caso del SVM, se presentaron desafíos en la correcta estructuración de los datos y en la comprensión del modelo matemático para lograr una clasificación adecuada.

Finalmente, la interpretación de los resultados gráficos también requirió un análisis detallado para validar el comportamiento esperado de los algoritmos.

3.7.3 Aplicaciones Industriales

Los algoritmos implementados en este trabajo tienen múltiples aplicaciones en la industria moderna. El filtro LMS es ampliamente utilizado en el procesamiento de señales, como en la cancelación de ruido en sistemas de audio, telecomunicaciones y equipos médicos (por ejemplo, electrocardiogramas).

Por otro lado, el uso de SVM es relevante en sistemas de clasificación y toma de decisiones, como el control de calidad en procesos industriales, detección de fallas en maquinaria, reconocimiento de patrones y sistemas de visión artificial.

La integración de estos algoritmos con sistemas embebidos permite el desarrollo de soluciones inteligentes en tiempo real, lo cual es fundamental en áreas como la automatización industrial, el mantenimiento predictivo y los sistemas autónomos.

3.8 Códigos y Video

Todo código y gráficos Del proyecto pueden encontrarse en Este enlace de Github

https://github.com/ShinigamiElite/Evaluacion1LMS-y-SVM.git